

Speculative decoding is a clever engineering trick used to make LLM generate text significantly faster *without losing any accuracy or changing the final output.*

The problem: autoregressive generation is slow

real constraints: if we want a LLM to generate a 100 word story, doing the math isn't the bottleneck, transferring the

massive weights back and forth is (memory-bandwidth bound)

Speculative decoding approach:

Say we still want to generate a 100 story using a (large) LLM

1. Draft

Let a much smaller, faster model generate the next k tokens

2. Verification

The large model verifies the k tokens at one pass

1. use masked self attention
2. do the same computation as generating sequentially
(but crucially, move the data only once)

3. Accept or reject

In Verification step, we calculate the probability for the k tokens in one pass.

Denote the probability for word i in large model as $p(w_i)$, in small model as $q(w_i)$, we then accept each word with probability:

$$\min(1, \frac{p(w_i)}{q(w_i)})$$

Note: 1. if a word is rejected, all words following it will be rejected.

2. when the word, e.g. w_j is rejected, we can still get the word at this location. because during verification, we calculate the next word prob distribution for w_j → this is not ideal though, because we wasted time on small model...

We sample each word with probability

across all words
$$\frac{\max(0, p(w_j) - q(w_j))}{\sum_j \max(0, p(w_j) - q(w_j))}$$

3. following point 2 if all tokens got rejected, we still get one token,

Why do we say Speculative decoding does not loss any accuracy?

The acceptance rate: $\min(1, \frac{p(w_i)}{q(w_i)})$ guarantees the output

probability of any word x will always be exactly $p(x)$

$\text{pr}(\text{word } i \text{ outputs})$

$= \text{pr}(\text{word } i \text{ is proposed by small model, and get accepted})$

$+ \text{pr}(\text{proposed word by small model get rejected, large model sample word } i)$

$$= q(w_i) \cdot \min(1, \frac{p(w_i)}{q(w_i)}) + \underbrace{(1 - \sum_j q(w_j) \cdot \min(1, \frac{p(w_j)}{q(w_j)}))}_{\substack{\downarrow \\ \text{probability of} \\ \text{rejecting all words}}} \cdot \frac{\max(0, p(w_i) - q(w_i))}{\sum_j \max(0, p(w_j) - q(w_j))}$$

notice that

$$p(x) = \min(p(x), q(x)) + \max(0, p(x) - q(x))$$

$$\Rightarrow 1 = \sum_j p(w_j) = \sum_j \min(p(w_j), q(w_j)) + \sum_j \max(0, p(w_j) - q(w_j))$$

$\Rightarrow$ probability of rejecting

$$1 - \sum_j q(w_j) \min(1, \frac{p(w_j)}{q(w_j)})$$

$$= 1 - \sum_j \min(q(w_j), p(w_j))$$

$$= \sum_j \max(0, p(w_j) - q(w_j))$$

$$\text{Therefore } \text{pr}(\text{word } i \text{ outputs}) = \min(q(w_i), p(w_i)) + \max(0, p(w_i) - q(w_i))$$

$$= p(w_i)$$

$\downarrow$
probability of drawing word w_i
from large model directly!